

CONNECT MARATHA — TECHNICAL REQUIREMENTS DOCUMENT (TRD)

****Product:**** Connect Maratha

****Document:**** Technical Requirements Document

****Version:**** 1.0

****Mobile Platform:**** Flutter / Dart

****Target Platforms:**** Android + iOS

****State Management:**** Riverpod

****Navigation:**** go_router

****Current Data Source:**** Local / Mock Data

****Future Backend:**** Node.js + TypeScript + NestJS

****Future Database:**** PostgreSQL

****Future ORM:**** Prisma

****Future API Client:**** Dio

****Current Backend Integration:**** NONE

****Architecture Style:**** Feature-first + Repository Pattern

1. PURPOSE

This document defines the technical architecture, development standards, project structure, application behavior, data architecture, UI architecture, testing requirements and future backend integration strategy for the Connect Maratha native Flutter application.

The application is being developed as a native mobile implementation of the existing Connect Maratha website.

The existing website is the functional and content source of truth.

The Flutter application must reproduce the major website capabilities while adapting the experience to a modern mobile-first interface.

2. CURRENT TECHNICAL SCOPE

The current phase focuses on:

- Flutter application development
- Native mobile UI
- Navigation
- Riverpod state management
- Local/mock data
- Feature architecture
- Reusable components
- Responsive layouts
- Android/iOS compatibility
- UI/UX implementation
- Feature flows
- Testing

The current phase **MUST NOT** connect to:

- Node.js backend
- PostgreSQL
- Production APIs
- WebSockets
- Firebase backend services
- Production authentication

The application must instead use local/mock repositories.

3. FUTURE TECHNICAL SCOPE

The future production architecture will connect:

Flutter



Riverpod



Repository



Dio



Node.js / NestJS



PostgreSQL

The future production architecture may also include:

- Redis
- Socket.IO/WebSockets
- Firebase Cloud Messaging
- Object storage
- Background jobs
- Admin platform
- Monitoring
- CI/CD

The master technical report recommends Node.js + TypeScript + NestJS, REST + WebSocket/Socket.IO, PostgreSQL + Prisma, Redis, FCM, object storage and Docker/CI/CD for production.

4. TECHNOLOGY STACK

4.1 Mobile

Technology:

Flutter

Language:

Dart

Targets:

- Android

- iOS

5. FLUTTER VERSION

Use the Flutter version already established in the project environment unless a deliberate upgrade is approved.

Current development environment:

Flutter 3.29.3

Dart 3.7.2

Do not upgrade Flutter/Dart unnecessarily during feature development.

If an upgrade becomes necessary:

1. Document the reason.
2. Verify package compatibility.
3. Run analyzer.
4. Run tests.
5. Build the application.
6. Test on the physical device.

6. STATE MANAGEMENT

Use:

Riverpod

Do NOT introduce:

- Provider
- Bloc
- GetX
- MobX
- Redux
- Another state-management framework

Riverpod should manage shared/application state.

Examples:

- Authentication state
- Current user
- Home data
- Community data
- Events
- Business data
- Heritage data
- Profile
- Referrals
- Notifications

7. LOCAL UI STATE

Not every state requires Riverpod.

Use local widget state for simple temporary UI state such as:

- Password visibility
- Selected tab
- Temporary search text
- Animation state
- Expanded/collapsed sections
- Temporary form state

Do not create a global provider for every boolean or text field.

8. NAVIGATION

Use:

go_router

Do NOT introduce another navigation package.

Navigation should support:

- Splash
- Login
- Register
- Forgot Password
- Home
- Community
- Member Profile
- Groups
- Group Details
- Business
- Business Profile
- Events
- Event Details
- Heritage
- History
- Forts
- Fort Details
- Warriors
- Services
- Jobs

- Messaging
- Referrals
- Profile
- Edit Profile
- Settings
- Notifications

Use named routes or structured route definitions.

Avoid scattered navigation strings throughout widgets.

9. ROUTE ARCHITECTURE

Recommended structure:

```
lib/  
  core/  
    routing/  
      app_router.dart
```

Routes should be centrally defined.

Example conceptual route tree:

/splash

/login

/register

/forgot-password

/home

/community

/community/member/:id

/community/groups

/community/groups/:id

/business

/business/:id

/events

/events/:id

/heritage

/heritage/history

/heritage/forts

/heritage/fort/:id

/heritage/warriors

/services

/jobs

/messages

/messages/:conversationId

/referrals

/profile

/profile/edit

/profile/qr

/settings

/notifications

10. FEATURE-FIRST PROJECT STRUCTURE

Recommended project structure:

lib/

|

├─ main.dart

|

├─ core/

- | | └─ constants/
- | | └─ routing/
- | | └─ theme/
- | | └─ utils/
- | | └─ errors/
- | | └─ widgets/
- |
- | └─ data/
- | | └─ models/
- | | └─ repositories/
- | | └─ datasources/
- |
- | └─ features/
 - |
 - | | └─ auth/
 - | | | └─ data/
 - | | | └─ domain/
 - | | | └─ presentation/
 - | |
 - | | └─ home/
 - | |
 - | | └─ community/
 - | |
 - | | └─ connections/
 - | |
 - | | └─ business/
 - | |
 - | | └─ events/
 - | |
 - | | └─ heritage/

```
|
|— services/
|
|— jobs/
|
|— messaging/
|
|— referrals/
|
|— profile/
|
└─ notifications/
```

11. CORE LAYER

The core layer contains functionality shared across features.

Core responsibilities

- Theme
- Colors
- Typography
- Spacing
- Routing
- Common widgets
- Error handling
- Constants
- Utilities

12. DESIGN SYSTEM

Create a centralized design system.

Recommended:

lib/core/theme/

app_colors.dart

app_theme.dart

app_typography.dart

app_spacing.dart

app_radius.dart

app_shadows.dart

Do not define random colors directly inside individual screens.

Do not create inconsistent padding values throughout the application.

13. COLOR SYSTEM

The design should support the Connect Maratha brand.

Primary visual language:

- Saffron

- Gold

- Cream
- White
- Deep charcoal
- Black

Use colors consistently.

Gold and saffron should be accents rather than being applied excessively.

Accessibility and contrast must be maintained.

14. TYPOGRAPHY

Typography must support:

- Marathi
- English

Use a consistent typography system.

Create reusable text styles for:

- Display
- Headline
- Title
- Body
- Caption
- Label
- Button

- Navigation

Do not manually define font sizes randomly in every widget.

15. SPACING SYSTEM

Create standardized spacing values.

Example:

4

8

12

16

20

24

32

40

48

Use the design system consistently.

16. REUSABLE WIDGETS

Create reusable components such as:

- AppButton

- AppOutlinedButton
- AppTextField
- AppDropdown
- AppCard
- AppSectionHeader
- AppChip
- AppAvatar
- AppLoading
- AppErrorState
- AppEmptyState
- AppBottomNavigation
- AppAppBar
- MemberCard
- BusinessCard
- EventCard
- HeritageCard
- GroupCard
- NotificationTile
- QRCard

Do not duplicate the same UI implementation across multiple screens.

17. DATA ARCHITECTURE — CURRENT PHASE

Current architecture:

UI



Riverpod

↓

Repository

↓

Local/Mock Data

Example:

CommunityScreen

↓

communityProvider

↓

CommunityRepository

↓

MockCommunityDataSource

18. REPOSITORY PATTERN

UI widgets must NOT directly access mock data.

Incorrect:

UI

↓

List<Map<String, dynamic>>

Correct:

UI

↓

Riverpod



Repository



Data Source

This allows the implementation to change later without rewriting UI.

19. FUTURE DATA ARCHITECTURE

Current:

Repository



MockDataSource

Future:

Repository



RemoteDataSource



Dio



Node.js API

The repository interface should remain stable where practical.

20. DATA MODELS

Create typed Dart models.

Avoid passing large amounts of raw:

Map<String, dynamic>

through the UI.

Examples:

User

Profile

CommunityPost

CommunityGroup

Business

Service

Event

Fort

Warrior

Job

Message

Conversation

Referral

Notification

Models should contain appropriate typed fields.

21. AUTHENTICATION — CURRENT PHASE

Use local/mock authentication.

Example:

MockAuthRepository

Possible state:

AuthenticationState

initial

loading

authenticated

unauthenticated

error

The application must behave as though authentication exists, but no real backend authentication should be implemented yet.

22. AUTHENTICATION — FUTURE

Future architecture:

Flutter

↓

Dio

↓

Node.js

↓

JWT access token

↓

Refresh token

Sensitive tokens should eventually be stored using secure storage.

Do not store production authentication secrets in normal shared preferences.

23. HOME TECHNICAL REQUIREMENTS

Home should be composed from reusable sections.

Example:

HomeScreen

└─ AppHeader

└─ Search

└─ MemberCard

└─ QuickActions

└─ UpcomingEvents

└─ CommunityActivity

└─ HeritageHighlights

└─ BottomNavigation

Avoid creating one extremely large Home widget.

Break sections into reusable widgets.

24. COMMUNITY TECHNICAL REQUIREMENTS

Community should support:

- Member discovery
- Search
- Filters
- Member cards
- Member profile
- Groups
- Group details
- Community feed
- Posts
- Comments
- Reactions
- Messaging entry points

Current implementation uses mock/local data.

Future implementation will use Node.js APIs.

25. COMMUNITY SEARCH

Search should support future filtering by:

- Name
- City
- Profession
- Business
- Skills
- Interests

Current phase:

Use local filtering.

Future phase:

Use backend search/pagination.

26. MEMBER PROFILE

Member profile model should support:

- ID
- Name
- Photo
- Location
- Profession
- Business
- Skills
- Interests
- About
- Membership

- Privacy
- QR/referral identifier

Sensitive/private information must not automatically be displayed.

27. QR SYSTEM

Every member can have a QR representation.

Current phase:

- Generate/display mock QR
- Display member information
- Scan UI
- Referral UI

Future phase:

- Server-generated secure token
- Referral attribution
- Member lookup
- Deep link
- Fraud prevention

DO NOT encode sensitive personal data directly into QR.

28. QR FLOW

Member QR:

User



My QR



QR displayed

Scan:

Scanner



QR Token



Local/mock resolver



Member Profile

Future:

Scanner



Deep Link



Node.js API



Member / Referral Resolution

29. REFERRAL SYSTEM

Current phase:

- Referral UI
- Referral QR
- Referral history
- Mock referral status

Future:

Node.js must control:

- Referral creation
- Attribution
- Qualification
- Duplicate prevention
- Referral status
- Commission calculation

Never trust client-side referral or earning values.

30. BUSINESS MODULE

Screens:

Business Directory

Business Search

Business Filters

Business Profile

Professional Profile

Enquiry

Current phase:

Mock business data.

Future:

Node.js API.

31. EVENTS MODULE

Screens:

Events

Event Details

Registration

Current:

Mock events.

Future:

Remote event API.

32. HERITAGE MODULE

Sections:

Heritage

History

Forts

Fort Details

Warriors

Important Dates

Knowledge/Culture

Use local assets where appropriate.

Heritage content should be structured rather than embedded as giant static images.

33. FORT DETAILS

Fort details should support:

- Hero image
- Title
- Marathi title where appropriate
- Overview
- History
- Statistics
- Attractions
- Trek information

- Travel information
- Location
- Favorite
- Share

Use reusable components.

34. MESSAGING

Current phase:

Build UI using mock conversations.

Future:

REST + WebSocket/Socket.IO.

Future capabilities:

- One-to-one chat
- Group chat
- Read status
- Delivery status
- Typing
- Presence
- Media
- Block
- Report

35. NOTIFICATIONS

Current phase:

Mock notification provider.

Future:

FCM + Node.js.

Notification model should support:

- ID
- Type
- Title
- Body
- Timestamp
- Read status
- Deep-link destination

36. PROFILE MODULE

Screens:

Profile

Edit Profile

My QR

Referrals

Settings

Profile should be composed from reusable components.

37. LOCAL STORAGE

For the current phase, local storage may be used for:

- Demo session
- User preferences
- Theme preference
- Onboarding state
- Small local settings

Do not store sensitive production secrets in insecure local storage.

Future authentication tokens should use secure storage.

38. IMAGE/ASSET MANAGEMENT

Existing website assets should be reviewed before adding duplicates.

Use:

assets/

images/

icons/

Flutter assets should be declared in pubspec.yaml.

Do not embed large images directly into Dart files.

Use appropriate image sizing/compression.

39. RESPONSIVE DESIGN

Do not hard-code:

width: 390

height: 844

Use:

- MediaQuery
- LayoutBuilder
- Flexible
- Expanded
- SafeArea
- Sliver widgets
- Constraints

The UI must work on different Android and iOS screen sizes.

40. MOBILE UX

Every screen must consider:

- SafeArea
- Status bar
- Bottom system navigation
- Keyboard
- Scrolling
- Touch targets
- Back navigation
- Loading
- Empty states
- Errors

41. FORMS

Forms must support:

- Validation
- Keyboard-safe scrolling
- Correct input types
- Error messages
- Loading state
- Success state
- Disabled submit state

Forms must not overflow when the keyboard opens.

42. ERROR HANDLING

Create centralized application error handling.

Possible categories:

- Validation error
- Network error
- Unauthorized
- Not found
- Server error
- Unknown error
- Offline

Current phase can simulate errors using mock repositories.

Future phase should map Node.js API errors into application-level errors.

43. LOADING STATES

Use:

- Skeleton loaders
- Progress indicators
- Button loading states
- Section-level loading states

Avoid freezing the entire UI unnecessarily.

44. EMPTY STATES

Every list-based screen must have an empty state.

Examples:

No Events

No Groups

No Members Found

No Messages

No Notifications

No Referrals

Empty states should explain what the user can do next.

45. OFFLINE STATES

Current phase:

Simulate offline/error states where useful.

Future phase:

Display:

- Connection unavailable
- Retry
- Cached content where appropriate

46. PERFORMANCE

Use:

- const widgets
- lazy lists
- ListView.builder
- SliverList where appropriate
- efficient rebuilds
- image optimization
- pagination-ready architecture

Avoid:

- unnecessary rebuilds
- giant widget trees
- huge unoptimized images
- excessive animations
- unnecessary global providers

47. ACCESSIBILITY

Support:

- readable text
- sufficient contrast
- semantic labels
- accessible touch targets
- screen reader compatibility
- dynamic text where practical

Do not communicate information only through color.

48. ANIMATION

Animations should be:

- subtle
- short
- meaningful
- performant

Use animation for:

- Navigation transitions
- Button feedback
- Tabs
- Cards
- Favorite action
- QR interactions

Avoid decorative animation that slows the app.

49. SECURITY — CURRENT PHASE

Even though the backend is not connected, do not design the application in a way that assumes the client can be trusted.

Do not put:

- API secrets
- Database credentials
- Private keys
- Production passwords

inside Flutter.

50. SECURITY — FUTURE

Production should implement:

- HTTPS/TLS
- JWT
- Refresh token security
- Server-side authorization
- Input validation
- Rate limiting
- CORS policy
- Secure headers

- Secret management
- Audit logs
- Dependency scanning
- Backup/restore

The master report specifies these as production security requirements.

51. FUTURE BACKEND

Recommended future backend:

Node.js

+

TypeScript

+

NestJS

Future modules:

AuthModule

UsersModule

ProfilesModule

CommunityModule

ConnectionsModule

BusinessModule

ReferralModule

WalletModule

MessagingModule

FeedModule

JobsModule

ServicesModule

NotificationsModule

LocationModule

MediaModule

SearchModule

AdminModule

AuditModule

HealthModule

52. FUTURE DATABASE

PostgreSQL

Future major entities:

users

user_profiles

roles

permissions

user_roles

sessions

devices

connections

businesses

business_members

posts

comments

reactions

groups
group_members
messages
conversations
participants
referrals
referral_events
wallet_accounts
ledger_entries
notifications
jobs
applications
services
enquiries
reports
moderation_actions
media_assets
audit_logs

Use:

- UUIDs/non-guessable identifiers
- Foreign keys
- Indexes
- Timestamps
- Soft deletion where appropriate
- Migration-controlled schema

53. FUTURE API

Future API should use versioning.

Example:

/api/v1/auth

/api/v1/users

/api/v1/community

/api/v1/businesses

/api/v1/events

/api/v1/heritage

/api/v1/referrals

Use:

- DTO validation
- Consistent error responses
- Pagination
- Request IDs
- OpenAPI/Swagger

54. API INTEGRATION LAYER

When backend integration begins:

UI

↓

Riverpod

↓

Repository



RemoteDataSource



Dio



Node.js

Do not call Dio directly from UI widgets.

55. PAGINATION

Future API-driven lists should be pagination-ready.

Applicable to:

- Members
- Posts
- Businesses
- Events
- Messages
- Notifications
- Jobs
- Services
- Groups

Current mock repositories should be structured so pagination can be introduced later.

56. REAL-TIME ARCHITECTURE — FUTURE

For real-time functionality:

Flutter



Socket.IO



Node.js/NestJS



Redis



PostgreSQL

Use WebSockets for:

- Chat
- Typing
- Presence
- Real-time events

REST remains responsible for reliable CRUD/state operations.

57. PUSH NOTIFICATIONS — FUTURE

Use:

Firebase Cloud Messaging

Flow:

Node.js



FCM



Android/iOS



Flutter notification handler



Deep link



Relevant screen

58. MEDIA — FUTURE

For images/videos/documents:

Flutter



Node.js



Object Storage

Potential storage:

- AWS S3

- Cloudflare R2

Use:

- Upload validation
- MIME validation
- Size validation
- Compression
- Thumbnail generation
- CDN where appropriate

59. LOCATION — FUTURE

Location should only be requested when needed.

Examples:

- Nearby businesses
- Nearby services
- Nearby members
- Event location
- Map features

Do not request background location unless genuinely required.

60. SEARCH — CURRENT

Use local filtering against mock data.

61. SEARCH — FUTURE

Initially use PostgreSQL search/indexing.

Later, if scale requires:

- OpenSearch
- Elasticsearch

Do not introduce a dedicated search engine prematurely.

62. TESTING STRATEGY

Testing must cover:

Unit Tests

- Models
- Repositories
- Providers
- Utilities
- Validation

Widget Tests

- Login

- Home
- Profile
- Community
- Business
- Events
- Heritage
- QR
- Referral

Integration Tests

- Login flow
- Registration flow
- Navigation
- Community discovery
- QR flow
- Referral flow

63. CURRENT DEVICE TESTING

Primary Android test device:

moto g45 5G

Device ID:

ZA222PGX3V

Run:

```
flutter run -d ZA222PGX3V
```

64. REQUIRED DEVELOPMENT COMMANDS

After changes:

```
dart format .
```

```
flutter analyze
```

```
flutter test
```

```
flutter build apk --debug
```

Do not claim testing was completed unless the commands actually ran successfully.

65. CODE QUALITY

Requirements:

- Meaningful names
- Small widgets
- Reusable components
- Typed models
- Separation of concerns
- No unnecessary duplication

- No business logic inside presentation widgets
- No direct database/API access from UI
- Proper disposal of resources
- Const constructors where appropriate

66. EXISTING WEBSITE ANALYSIS RULE

Before implementing any feature:

1. Inspect the existing website.
2. Identify the page/functionality.
3. Identify user flow.
4. Identify data.
5. Identify actions.
6. Identify related screens.
7. Convert desktop experience to mobile.
8. Implement native Flutter UI.
9. Use mock/local data.
10. Test.
11. Preserve feature behavior.

DO NOT blindly invent new functionality.

67. UI IMPLEMENTATION RULE

Generated UI reference images are visual references only.

Do not implement an entire screen as a single image.

Use:

- Flutter widgets
- Cards
- Containers
- Text
- Images
- Icons
- Lists
- Slivers
- Buttons
- Inputs

The UI must remain interactive and responsive.

68. WEBSITE IMMUTABILITY RULE

The existing website must NOT be modified during Flutter development.

Only the Flutter project should be changed.

The website is:

SOURCE / REFERENCE

The Flutter project is:

ACTUAL DEVELOPMENT TARGET

69. ENVIRONMENT SEPARATION

Current:

LOCAL DEVELOPMENT



Flutter



Mock Data

Future:

DEVELOPMENT



Development API



Development Database

STAGING



Staging API



Staging Database

PRODUCTION



Production API



Production Database

Never use production credentials inside development builds.

70. CONFIGURATION

Use environment-specific configuration.

Future examples:

Development:

api-dev.connectmaratha.com

Staging:

api-stage.connectmaratha.com

Production:

api.connectmaratha.com

Current phase does not require real API URLs.

71. LOGGING

Current:

Use debug logging only where necessary.

Do not log:

- Passwords
- OTPs
- Tokens
- Private user information

Future production logging must also avoid sensitive data.

72. DEEP LINKS — FUTURE

Deep links will eventually be useful for:

- Referral links
- Member profiles
- Events
- Businesses
- Services
- Posts
- Groups
- Notifications

Example:

<https://connectmaratha.com/join?ref=XXXXX>

73. QR + DEEP LINK ARCHITECTURE

Future:

QR



Deep Link



Flutter



Referral Token



Node.js



Referral Resolution



Registration

This should be designed during the UI phase but implemented against the backend later.

74. FUTURE ADMIN PLATFORM

The mobile app should not attempt to contain the entire administration system.

Future administration can be implemented separately using:

Next.js + TypeScript

Admin areas:

- Users
- Roles
- Permissions
- Reports
- Moderation
- Business approval
- Referrals
- Wallet
- Notifications
- Analytics
- Configuration
- Audit logs

75. DEPLOYMENT — CURRENT

Current development target:

Android debug APK.

The application should be tested on the physical Android device.

iOS development should remain compatible with Flutter architecture.

76. PRODUCTION DEPLOYMENT — FUTURE

Android:

flutter build appbundle --release

Production artifact:

AAB

iOS:

Flutter + Xcode + Apple Developer + App Store Connect

Production deployment will require:

- Signing
- App IDs
- App Store metadata
- Privacy policy
- Data Safety
- Account deletion
- Testing
- Review
- Production API
- HTTPS

77. GOOGLE PLAY REQUIREMENTS

Before release:

- Final application ID

- Signing
- Play App Signing
- Target API requirement
- AAB
- Store listing
- Privacy policy
- Data Safety
- Content rating
- Account deletion
- Reviewer access
- UGC moderation
- Testing

The uploaded master report notes the Android 16/API 36 target requirement applicable to submissions from August 31, 2026 and other current Play requirements.

78. APPLE RELEASE REQUIREMENTS

Before iOS release:

- macOS
- Xcode
- Apple Developer account
- Bundle ID
- App Store Connect
- Signing
- TestFlight
- App Review

79. CI/CD — FUTURE

Future CI/CD should automatically:

1. Run formatting checks.
2. Run analyzer.
3. Run tests.
4. Build Flutter artifacts.
5. Build backend.
6. Run backend tests.
7. Validate migrations.
8. Deploy to staging.
9. Support controlled production deployment.

GitHub Actions may be used.

80. MONITORING — FUTURE

Recommended:

- Sentry
- Application metrics
- API latency monitoring
- Crash monitoring
- Notification monitoring
- Database monitoring
- Server health monitoring

81. BACKUP — FUTURE

Production PostgreSQL must have:

- Automated backups
- Backup retention
- Restore testing
- Migration backup procedures

Never perform risky production database migrations without a rollback/backup strategy.

82. SECURITY REQUIREMENTS

Production must include:

- HTTPS/TLS
- Secure authentication
- Server-side authorization
- Input validation
- Rate limiting
- Secure token handling
- Secret management
- SQL injection protection
- Secure file upload
- Audit logging
- Dependency vulnerability scanning

- Backup/restore testing

83. PRIVACY

The application should collect only information necessary for the requested feature.

Location should only be requested when necessary.

Personal information should have appropriate visibility controls.

Do not put sensitive information into QR codes.

84. CURRENT PHASE ACCEPTANCE CRITERIA

A feature is considered complete when:

1. Corresponding website functionality has been analyzed.
2. Flutter screen exists.
3. Navigation works.
4. Mock/local data works.
5. Riverpod state is implemented where necessary.
6. Loading state exists where applicable.
7. Empty state exists.
8. Error state exists where applicable.
9. Responsive layout works.
10. No overflow occurs.
11. No major analyzer errors.

12. Device testing is completed.

85. FINAL ARCHITECTURE

CURRENT:

Flutter



Riverpod



Repository



Local/Mock Data

FUTURE:

Flutter



Riverpod



Repository



Dio



Node.js + TypeScript + NestJS



REST + WebSocket



PostgreSQL + Prisma

Supporting infrastructure:

Redis

FCM

Object Storage

BullMQ

Docker

CI/CD

Monitoring

86. MOST IMPORTANT DEVELOPMENT RULE

DO NOT BUILD A COMPLETELY DIFFERENT PRODUCT.

The Flutter application must be based on the existing Connect Maratha website.

Use:

Existing Website

↓

Existing Features

+

Existing Content

+

Existing User Flows

↓

Native Mobile UX

↓

Flutter



Riverpod



Mock/Local Data



Future Node.js API



PostgreSQL

The goal is:

EXISTING CONNECT MARATHA

+

NATIVE MOBILE EXPERIENCE

+

PREMIUM UI/UX

+

CLEAN ARCHITECTURE

+

BACKEND-READY DESIGN

87. DEVELOPMENT PHASE SUMMARY

PHASE 1

Foundation

- Flutter

- Riverpod

- go_router

- Theme
- Core widgets

PHASE 2

Authentication UI

- Splash
- Login
- Register
- Forgot Password
- Mock auth

PHASE 3

Core Member Experience

- Home
- Profile
- Edit Profile
- QR
- Community
- Groups

PHASE 4

Content

- Heritage
- History
- Forts
- Warriors
- Events

PHASE 5

Networking

- Business

- Services
- Connections
- Referrals

PHASE 6

Communication

- Feed
- Messages
- Notifications

PHASE 7

Backend Integration

- Node.js
- API
- PostgreSQL
- Authentication
- Real data

PHASE 8

Production Hardening

- Security
- Testing
- Monitoring
- Performance
- Backups

PHASE 9

Store Release

- Android
- iOS
- Play Store

- App Store

88. FINAL TECHNICAL PRINCIPLE

The Flutter application must be:

NATIVE

SCALABLE

RESPONSIVE

ACCESSIBLE

MAINTAINABLE

BACKEND-READY

PRODUCTION-READY

But during the current development phase:

NO REAL BACKEND CONNECTION.

Focus on building the complete native mobile experience using local/mock data while preserving the functionality of the existing Connect Maratha website.